

Arabic Post-OCR Correction with LLMs

Can LLMs bridge the performance gap between open-source and closed-source Arabic OCR?

Progress Report

Mohamed Sabry · May 2026

Model: Qwen3-4B-Instruct-2507 · Datasets: PATS-A01 (typewritten) + KHATT (handwritten)

Research Question & Motivation

The Problem

- Arabic OCR (Qaari) is open-source & accessible
- But makes systematic character-level errors:
 - Similar-shape confusions (ث / ت / ب)
 - Missing/extra diacritics
 - Punctuation & spacing artefacts
- Closed-source VLMs are far more accurate but expensive and API-dependent

Why It Matters: Digitising Arabic manuscripts, newspapers, and historical archives at low cost requires accurate open-source OCR — LLM post-correction is a practical bridge.

Research Question

Can a small open-source LLM (4B) post-process Qaari OCR output to approach the accuracy of closed-source VLMs?

Hypothesis

LLMs can exploit linguistic context to fix OCR errors cheaply — making open-source OCR competitive.

Approach: 8 experiments test progressively richer prompting strategies (zero-shot → prompt engineering → RAG)

Datasets & OCR Baseline (Phase 1)

Two domains: typewritten (PATS-A01) vs handwritten (KHATT)

PATS-A01 (Typewritten / Synthetic)

8 Arabic fonts · ~2766 pages/font · 85/15 train/val split

Font	Val CER	Val WER
Naskh	0.86%	0.92%
Thuluth	1.02%	1.07%
Arial	1.06%	1.08%
Akhbar	1.14%	1.11%
Simplified	1.32%	1.33%
Traditional	1.18%	1.22%
Andalus	1.54%	1.66%
Tahoma	2.39%	2.35%
PATS avg	1.31%	1.34%

KHATT (Handwritten / Real)

Writers of varied styles · 226 validation pages

CER: 5.66% | WER: 7.25% | Runaway: 17.7%

Key Observations:

KHATT CER is 4.3x higher than PATS average

Runaway samples (Qaari repetition bug):

KHATT: 17.7%, Tahoma: 10.4%

Most PATS fonts: 3.5 – 6%

Tahoma is hardest typewritten font (CER 2.39%)

Naskh is easiest typewritten font (CER 0.86%)

Two very different correction challenges:

PATS — already accurate, LLM risks over-correction

KHATT — high noise, LLM has more to gain

8-Phase Experimental Design

Each phase isolates one variable; all compare to Phase 2 (zero-shot) baseline

Phase	Name	What it tests	Key signal added
1	Baseline	Raw Qaari OCR quality, error taxonomy	None (reference)
2	Zero-Shot LLM	Can vanilla LLM fix OCR without any extra context?	None (base prompt p2v2)
3	OCR-Aware	Does confusion-matrix knowledge help?	Top-10 char confusions + LLM failure filter
4	Self-Reflective	Do error-pattern warnings reduce over-correction?	Training failure pairs + overcorrection alerts
5	CAMeL Validation	Does morphological revert save over-corrected words?	CAMeL morph check + known-OC revert
6	Combinations	Do Phase 3+4 signals synergize?	Confusion + self-reflective fused prompt
7	RAG (BM25)	Does per-sample retrieval of similar corrections help?	Top-5 (OCR→GT) pairs from training index
8	Error-Sig RAG	Does structural similarity retrieval beat surface?	CAMeL + confusion-matrix error signature

Main Results — All Phases

CER = Character Error Rate · WER = Word Error Rate · Lower is better

Validation set · Diacritics stripped before evaluation · Model: Qwen3-4B-Instruct-2507

Phase	PATS avg CER	PATS avg WER	KHATT CER	KHATT WER
Phase 1 (OCR Baseline)	1.31%	1.34%	5.66%	7.25%
Phase 2 (Zero-Shot)	1.32%	1.38%	3.21%	4.53%
Phase 3 (OCR-Aware)	1.47%	1.51%	3.33%	3.59%
Phase 4 (Self-Reflective)	1.45%	1.49%	3.02%	3.44%
Phase 6 (Conf+Self)	1.53%	1.57%	3.30%	3.87%
Phase 7 (RAG BM25)	1.29%	1.32%	2.77%	3.28%
Phase 8 (Error-Sig RAG)	1.44%	1.48%	3.18%	3.65%
Phase 5 (CAMEL revert)	see note*	—	3.26%	4.64%

* Phase 5 (CAMEL): only normal-subset metrics reported (excludes runaway samples).

Key Finding 1 — RAG (BM25) Is the Best Overall Strategy

KHATT CER reduction
vs OCR baseline

-51%

5.66% → 2.77%

KHATT CER reduction
vs Zero-Shot (P2)

-14%

3.21% → 2.77%

KHATT WER reduction
vs OCR baseline

-55%

7.25% → 3.28%

PATS avg CER
(only phase to beat P2)

1.29%

P1=1.31%, P2=1.32%

Why RAG (BM25) Works

Domain-matched examples

Top-5 (OCR→GT) pairs from Phase 2 training corrections — same domain, same font family, same error patterns.

Grounded few-shot

Instead of abstract prompting, the model sees concrete examples of what the correction should look like.

Reduces hallucination

Ground-truth anchors prevent the model from inventing changes unsupported by training evidence.

BM25 char n-grams

Character-level matching captures OCR-specific confusable character patterns rather than word-level semantics.

Key Finding 2 — Prompt Engineering Regresses on Typewritten Text (PATs)

Phases 3, 4, 6 all increase PATs CER compared to zero-shot (Phase 2). More prompt context = more incorrect changes on already-accurate text.

Phase	PATs avg CER	vs Zero-Shot	Interpretation
Phase 2 — Zero-Shot (baseline)	1.32%	—	Reference
Phase 3 — OCR-Aware	1.47%	+0.15pp	Confusion matrix → over-corrects
Phase 4 — Self-Reflective	1.45%	+0.13pp	Failure warnings → more edits
Phase 6 — Conf+Self	1.53%	+0.21pp	Combined signals → worst of both

Root Cause: Over-Correction at Low Error Rate

The floor effect

PATs CER is already 0.86%–1.54%. At this level, any non-trivial correction strategy introduces more errors than it fixes. Signal-to-noise ratio is very low.

More context = more edits

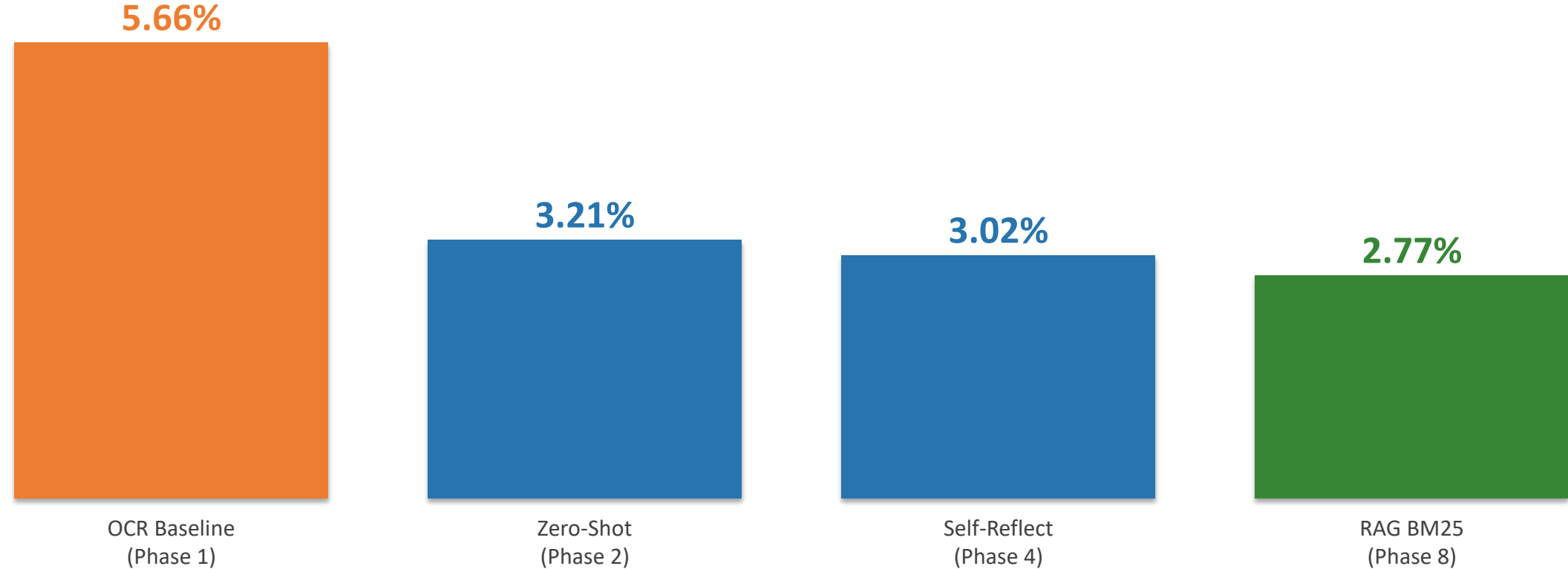
Confusion matrices and failure examples implicitly encourage the model to make changes — even when the OCR text is already correct.

Exception: Tahoma

CER 2.39% is high enough for LLM to help (Phase 2: 2.70%), suggesting a per-dataset difficulty threshold below which LLM correction is net-harmful.

Key Finding 3 — LLM Post-Correction Strongly Benefits Handwritten Text (KHATT)

KHATT CER Progression (WER follows similar pattern: 7.25% → 3.28%)



Total CER reduction: OCR → RAG = 5.66% → 2.77% = 51% relative improvement (WER: 7.25% → 3.28% = 55% relative improvement)

KHATT is the real target domain: high error rate gives LLM enough signal to correct without excessive false positives. RAG amplifies this further via domain-matched examples.

Per-Font Breakdown — PATS-A01

Not all fonts benefit equally; Tahoma uniquely resists LLM correction

Font	P1 CER (OCR)	P2 CER (ZeroShot)	P8 CER (RAG)	Best Phase	Observation
Naskh	0.86%	1.01%	0.90%	P1	Already near-perfect; LLM hurts
Thuluth	1.02%	0.83%	0.75%	P8 ★	RAG most effective here
Arial	1.06%	1.21%	1.10%	P1	LLM degrades accuracy
Akhbar	1.14%	0.87%	0.89%	P2	ZeroShot sufficient
Traditional	1.18%	1.27%	1.30%	P1	Slight LLM regression
Simplified	1.32%	1.61%	1.26%	P8	RAG recovers from P2 regression
Andalus	1.54%	1.08%	1.28%	P2	ZeroShot helps most
Tahoma	2.39%	2.70%	2.81%	P1	No phase beats OCR baseline!
PATS avg	1.31%	1.32%	1.29%	P8	RAG is only phase to beat P1

Analysis — The Error-Rate Threshold Effect

LLM correction is beneficial only when OCR error rate exceeds a critical threshold

Hypothesis: below ~2% CER, LLM correction introduces more errors than it fixes.

HARMFUL ZONE

CER < ~1.5%

LLM makes text worse
Naskh, Arial, Traditional
Best strategy: return unchanged

MIXED ZONE

CER 1.5 – 2.5%

Depends on strategy
Andalus, Simplified, Tahoma
RAG and ZeroShot can help

BENEFICIAL ZONE

CER > ~2.5%

LLM significantly improves
KHATT (5.66%), Tahoma (2.39%)
RAG gives best results

Implication for thesis:

1. Dataset difficulty must be reported alongside CER/WER improvements.
2. A selective-correction strategy (detect high-error samples, then apply LLM) could unlock gains on PATS.
3. Phase 2 (zero-shot) already achieves diminishing returns — advanced methods mainly help KHATT.

Next Step 1 — Conservative Prompt (p2v3)

Hypothesis: an even stricter 'do-nothing-when-unsure' policy will reduce PATS over-correction

Current prompts

p2v1 (aggressive):

"Fix OCR errors in this Arabic text. Return only the corrected text."

p2v2 (conservative, current):

"You are an Arabic OCR corrector. If the text looks correct, return it unchanged. Only fix clear OCR errors such as wrong characters, missing characters, or obvious character confusions. Do not rephrase or paraphrase."

Problem with p2v2:

Still triggers too many edits on PATS fonts (Arial, Traditional, Naskh) where OCR is already highly accurate. CER increases vs baseline.

Proposed: p2v3 (ultra-conservative)

"You are an Arabic OCR post-processor. Your ONLY job is to fix unambiguous character-substitution errors caused by the OCR scanner — where a character was clearly misread due to visual similarity. Do NOT change: word choice, spelling variants, diacritics, punctuation, spacing, or any word that could be a valid Arabic word. If you are unsure whether something is an OCR error or correct Arabic, leave it unchanged. Return ONLY the corrected text, nothing else."

Key differences from p2v2:

- Explicit list of what NOT to change
- Uncertainty → do nothing (conservative default)
- 'Valid Arabic word' test blocks grammar 'fixes'
- Targets only visual confusion errors

Expected: PATS CER holds at baseline; KHATT may trade recall for precision

Next Step 2 — Larger LLM Model

Hypothesis: a larger model has better Arabic orthographic knowledge → fewer over-corrections

Current Model

Model:	Qwen3-4B-Instruct-2507
Parameters:	4 Billion (dense)
Platform:	Kaggle T4 GPU (~15 GB VRAM)
Speed:	~3–5 samples / second
Arabic skill:	Good — multilingual training data
Limitation:	Borderline word-validity decisions; limited orthographic certainty → over-corrects

Expected Impact of Larger Model

PATS:	Better morphological knowledge → fewer false positives → CER closer to baseline
KHATT:	Stronger context modelling → better error detection
RAG:	Higher reasoning capacity → better use of examples

Candidate Larger Models

Qwen3-14B

Size: 14B dense · Platform: Kaggle A100
Recommended: same family as current;
3.5x parameters, direct comparison

Qwen3-30B-A3B (MoE)

Size: 30B / 3B active · Platform: Kaggle A100
Efficient MoE; near 14B quality
at fraction of compute

Qwen3-32B

Size: 32B dense · Platform: 2x Kaggle A100
Largest accessible Qwen3;
may need quantization

Llama-3.3-70B-Q4

Size: 70B (4-bit) · Platform: Colab Pro+
Strong Arabic; higher cost;
different architecture

Start with Qwen3-14B — same tokenizer, minimal code changes

Next Steps & Timeline

Two targeted experiments before thesis finalisation

1

Implement p2v3 (ultra-conservative prompt)

Add prompt version to PromptBuilder.build_zero_shot(version='v3')
Update config to allow phase2.prompt_version: p2v3
Run export → infer → analyze on all 9 datasets

~1 week

2

Run p2v2 + p2v3 on Qwen3-14B

Use same 3-stage pipeline (export → infer on Kaggle A100 → analyze)
Compare: 4B-p2v2, 4B-p2v3, 14B-p2v2, 14B-p2v3 (2x2 design)
Also run Phase 8 (RAG) with 14B to measure retrieval interaction

~2 weeks

3

Statistical significance testing

Run bootstrap CIs + paired t-test for best results vs Phase 2 baseline
Bonferroni correction for multiple comparisons across 9 datasets
Also: Wilcoxon signed-rank for non-normal per-sample CER distributions

~3 days

Summary

LLMs work well on high-error Arabic OCR

KHATT: 51% CER reduction (5.66% → 2.77%) using RAG + Qwen3-4B

Prompt engineering regresses on low-error text

PATS phases 3/4/6 increase CER vs zero-shot — over-correction at low noise floor

RAG (BM25) is the best strategy overall

Domain-matched training examples outperform all pure-prompt approaches

Next: conservative prompt + Qwen3-14B

p2v3 to reduce false positives; 14B for stronger Arabic orthographic knowledge